Инструкция для видеопрезентации LabFlow Planner

Цель записи - за 5-7 минут показать, какую задачу решает проект, как работает API, как устроена ключевая часть кода и какой результат дает pylint.

Подготовка перед записью

1. Открыть терминал в корне проекта.
2. Убедиться, что виртуальное окружение создано и зависимости установлены:

```
source .venv/bin/activate
pip install -r requirements.txt
```

3. Пересоздать локальную базу, если нужна чистая демонстрация:

```
rm -f labflow.db
```

4. Запустить сервис:

```
uvicorn app.main:app --reload
```

5. Открыть в браузере:

- http://127.0.0.1:8000/
- http://127.0.0.1:8000/docs

6. Открыть в редакторе файл `app/services/planner.py`.
7. Открыть в терминале отдельную вкладку для команды:

```
pylint app
```

8. Закрыть лишние приложения, уведомления и вкладки.
9. В Zoom включить камеру, микрофон, демонстрацию всего экрана и запись.

Тайминг на 5-7 минут

0:00-0:40 - Вступление и задача

Сказать:

> Проект называется LabFlow Planner. Это REST-сервис на FastAPI для небольшой лаборатории, где нужно вести проекты, образцы, приборы и запуски анализов. Главная бизнес-задача - автоматически построить план анализов с учетом приоритетов, дедлайнов, требуемого метода и дневной мощности оборудования.

Показать главную страницу `http://127.0.0.1:8000/`.

Отметить:

- есть визуальная стартовая страница;
- Swagger доступен через кнопку;
- ReDoc доступен для просмотра схемы API.

0:40-1:20 - Архитектура данных

Открыть Swagger или ReDoc.

Сказать:

> В проекте используется реляционная SQLite-БД. При запуске приложение автоматически создает файл `labflow.db`. Основные связанные таблицы: пользователи, проекты, образцы, приборы и запуски анализов. Проект принадлежит пользователю, образцы относятся к проекту, а запуск связывает образец с прибором.

Показать группы эндпоинтов:

- `auth`;
- `users`;
- `projects`;
- `equipment`;

- `samples`;
- `analysis runs`;
- `planner`.

1:20-3:40 - Демонстрация функциональности API

Открыть `/docs`.

Шаг 1. Регистрация

Выполнить `POST /auth/register`:

```
{
  "email": "manager@example.com",
  "username": "labmanager",
  "password": "StrongPass123",
  "role": "manager"
}
```

Сказать:

> Регистрация создает пользователя, пароль хранится не в открытом виде, а как PBKDF2-SHA256-хэш с солью.

Шаг 2. Получение токена

Выполнить `POST /auth/token`:

- username: `manager@example.com`
- password: `StrongPass123`

Скопировать `access_token`, нажать `Authorize`, вставить:

```
Bearer <полученный токен>
```

Сказать:

> Дальше защищенные эндпоинты работают через JWT Bearer-токен. Без токена, например, `/users/me` вернет 401.

Шаг 3. Создание проекта

Выполнить `POST /projects`:

```
{
  "title": "Enzyme stability sprint",
  "description": "Short-cycle screening for formulation candidates",
  "deadline": "2026-05-20",
  "status": "active"
}
```

Запомнить `id` проекта.

Шаг 4. Создание прибора

Выполнить `POST /equipment`:

```
{
  "name": "qPCR station",
  "method": "qPCR",
  "capacity_per_day": 8,
  "status": "active",
  "calibration_due": "2026-06-30"
}
```

Шаг 5. Создание двух образцов

Выполнить `POST /samples` для первого образца:

```
{
  "project_id": 1,
  "code": "ENZ-LOW",
  "material_type": "enzyme",
  "required_method": "qPCR",
  "priority": 1,
  "estimated_hours": 4,
  "deadline": "2026-05-10",
  "status": "new"
}
```

Выполнить `POST /samples` для второго образца:

```
{
  "project_id": 1,
  "code": "ENZ-HIGH",
  "material_type": "enzyme",
  "required_method": "qPCR",
  "priority": 5,
  "estimated_hours": 4,
  "deadline": "2026-05-10",
  "status": "new"
}
```

Шаг 6. Расчет плана

Выполнить `POST /planner/optimize`:

```
{
  "sample_ids": [1, 2],
  "horizon_days": 3,
  "start_date": "2026-05-01",
  "include_existing_runs": true
}
```

Сказать:

> В ответе видно, что образец с приоритетом 5 попадает в план раньше образца с
>
> приоритетом 1. Алгоритм выбрал совместимый прибор `qPCR station` и не превысил
>
> дневную мощность 8 часов.

Шаг 7. Сохранение плана

Выполнить `POST /planner/commit` с тем же телом.

Затем выполнить `GET /runs`.

Сказать:

> После сохранения появляются записи запусков анализов. Это уже не просто
>
> расчет, а изменение связанных таблиц БД.

3:40-5:10 - Техническое решение и фрагмент кода

Открыть `app/services/planner.py`.

Показать функции:

- `_load_samples`;
- `_initial_capacity`;
- `_score_sample_on_day`;
- `_best_slot`;
- `build_plan`.

Сказать:

> Здесь находится алгоритмическая часть. Сначала сервис получает только образцы
>
> текущего пользователя, исключая завершенные и отмененные. Затем загружает
>
> активное оборудование и строит таблицу доступной мощности по дням. Если уже
>
> есть запланированные запуски, они уменьшают доступную мощность.

Показать `_score_sample_on_day`.

Сказать:

> Функция оценки дает плюс за высокий приоритет, штрафует за просрочку дедлайна
>
> и небольшой штраф дает за более поздний день в горизонте. После этого
>
> `_best_slot` перебирает дни и приборы, оставляя только совместимые варианты,
>
> где хватает мощности.

Показать `build_plan`.

Сказать:

> В результате возвращаются два списка: `scheduled` для назначенных образцов и
>
> `unscheduled` для тех, которые нельзя поставить в план. Например, если нет
>
> прибора с нужным методом, причина будет явно указана в JSON.

5:10-5:50 - Тесты

Открыть терминал и выполнить:

```
pytest
```

Сказать:

> Тесты используют FastAPI TestClient. Проверяются регистрация, вход, защищенный
>
> эндпоинт, CRUD-сценарии, валидация, изоляция данных по пользователю,
>
> оптимизация плана и сохранение запусков. Покрытие выше требуемых 70%.

5:50-6:30 - Pylint

Выполнить:

```
pylint app
```

Сказать:

> Pylint проверяет папку `app`. Текущий результат - 10.00 из 10. Отчет для сдачи
>
> сохранен командой `pylint app > pylint.txt`.

6:30-7:00 - Завершение

Сказать:

> Проект выполняет требования: есть FastAPI REST API, реляционная БД с пятью
>
> связанными таблицами, полный CRUD, JWT-авторизация, Pydantic-валидация,
>
> алгоритмическая бизнес-логика, тесты TestClient, README и файл `pylint.txt`.

Остановить запись.

Демо-аккаунт для быстрой проверки

Можно просто зайти в этот аккаунт и посмотреть на работоспособность сайта:

- username: `labcurator`
- пароль: `LabPlan2026`

В аккаунте уже есть проект, активные приборы, образцы и запланированные анализы. Вводить данные можно прямо на главной странице приложения.

Частые проблемы при записи

Регистрация возвращает 409

Пользователь уже создан. Удалить `labflow.db`, перезапустить сервис и повторить демонстрацию.

Authorize не принимает токен

Проверить, что вставлен формат:

```
Bearer <token>
```

Если Swagger сам добавляет `Bearer`, вставить только сам токен.

`POST /samples` возвращает 404

Проверить, что в поле `project_id` указан реальный id проекта, созданного под текущим токеном.

`planner/optimize` возвращает `unscheduled`

Проверить, что:

- `required_method` у образца совпадает с `method` у прибора;
- прибор имеет статус `active`;
- `estimated_hours` образца не больше `capacity_per_day` прибора;
- горизонт планирования достаточно длинный.

Pylint не найден

Активировать окружение и установить зависимости:

```
source .venv/bin/activate
pip install -r requirements.txt
```